MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and Surname: MuhammadSaad Bariwala

Student ITS No: 402412192

Qualification: BSc in IT **Year of Study:** 3 **Semester:** 2

Assignment due date: 30-09-2026 **Date submitted:** 30-09-2026

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:
I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: Saad **Date:** 30-092026

Richfield Graduate Institute of Technology (Pty) Ltd is registered with the Department of Higher Education & Training as a Private Higher Education Institution under the Higher Education Act, 1997, Registration Certificate No. 2000/HE07/008.

2. TABLE OF CONTENTS

1.Cover Page & Declaration of Originality

2.Table of Contents

3.Introduction

- 3.1 Problem Statement
- 3.2 Target Audience & Core Value Proposition

4.System Design

- 4.1 Screen Flow & Wireframe Architecture Diagram
- 4.2 Database ER Diagram & Schema Design

5.Screenshots of Every Output & Core Function

- 5.1 Pantry Inventory Screen (PantryListActivity)
- 5.2 Add & Edit Ingredient Form (AddEditIngredientActivity)
- 5.3 Form Input Validation Error State
- 5.4 Delete Confirmation Dialog
- 5.5 Suggested Recipes Screen - 100% Strict Matches (SuggestedRecipesActivity)
- 5.6 Suggested Recipes Screen - "Almost There" (Missing 1 Item)
- 5.7 Zero-Match Empty State View
- 5.8 Recipe Collection & Category Filter (AllRecipesActivity)
- 5.9 Recipe Detail & Ingredient Checklist (RecipeDetailActivity)
- 5.10 "I Cooked This Recipe" Ingredient Deduction
- 5.11 Settings & Database Reset Screen (SettingsActivity)

6.Key Code Snippets & Technical Explanations

- 6.1 Core Strict-Matching Algorithm
- 6.2 Name Normalization & Unit Conversion Engine (IngredientMatcher.java)
- 6.3 SQLite Relational Database Management (DatabaseHelper.java)
- 6.4 Automatic Ingredient Deduction Upon Cooking (DatabaseHelper.java)

7.Challenges Encountered & Solutions Applied

- 7.1 Challenge 1: String Matching & Pluralization Errors
- 7.2 Challenge 2: Measurement Unit Incompatibilities
- 7.3 Challenge 3: Dark Mode Visibility & System Bar Overlaps

8.Conclusion & Reflection

- 8.1 Summary of Accomplishments
- 8.2 Future Enhancements

9.Reference List (Harvard Referencing Style)

3. INTRODUCTION

3.1 Problem Statement

Food waste is one of the most critical household economic and environmental challenges globally. Millions of tons of edible food are discarded annually because consumers buy unnecessary groceries while ignoring leftover ingredients already sitting in their refrigerators, cupboards, or pantries. A major reason for this behavior is that home cooks often find it difficult to identify complete recipes that can be prepared using solely the miscellaneous items available at home.

Existing recipe applications typically fail in this regard because they suggest recipes requiring additional grocery shopping trips or missing key ingredients.

3.2 Target Audience & Core Value Proposition

The SmartPantry application is a native Android application developed strictly in Java to solve household food waste. The core value proposition of SmartPantry is its Strict-Matching Rule:

"A recipe is suggested to the user ONLY if 100% of its required ingredients are already present in the user's pantry in at least the required quantity — requiring zero extra grocery shopping."

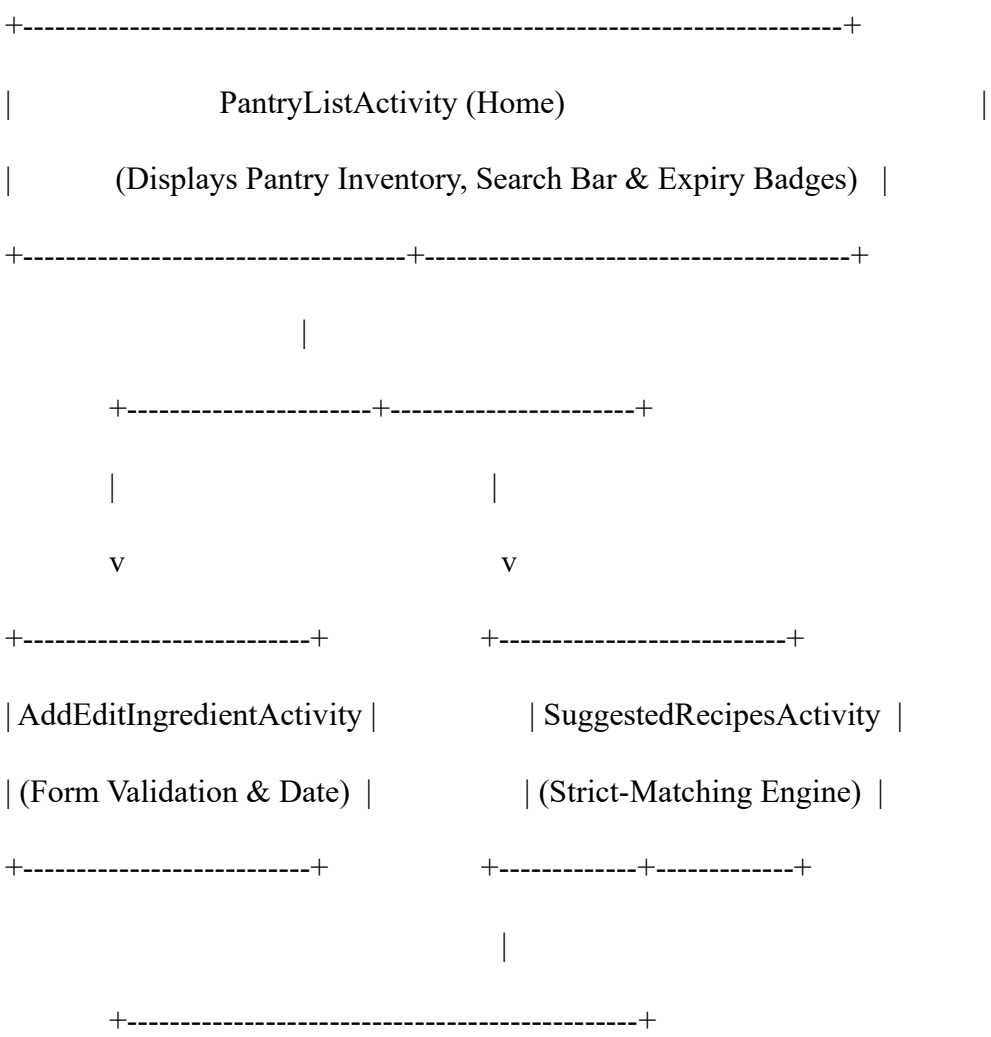
The target audience includes:

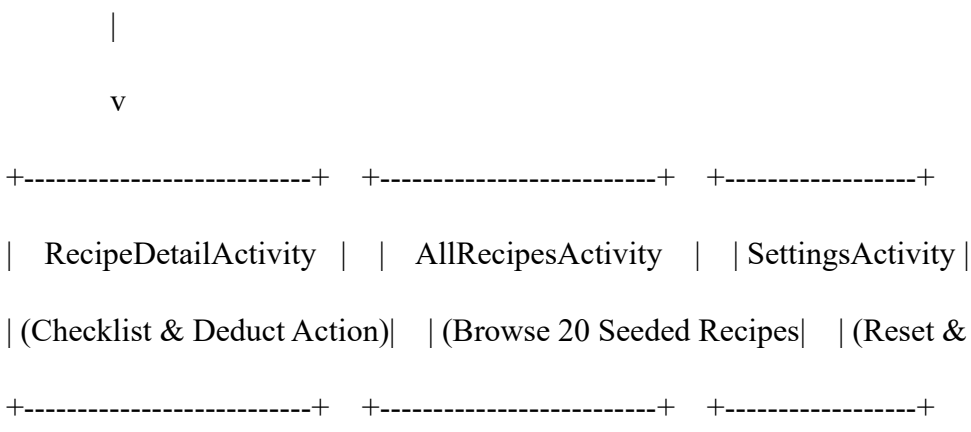
- Busy Professionals and Families: Seeking to reduce weekly grocery bills by utilizing existing leftover ingredients.
- Students and Cost-Conscious Individuals: Wanting quick meal ideas based strictly on what is available in their kitchen.
- Environmentally Conscious Consumers: Aiming to minimize household food wastage

4. SYSTEM DESIGN

4.1 Screen Flow & Wireframe Architecture Diagram

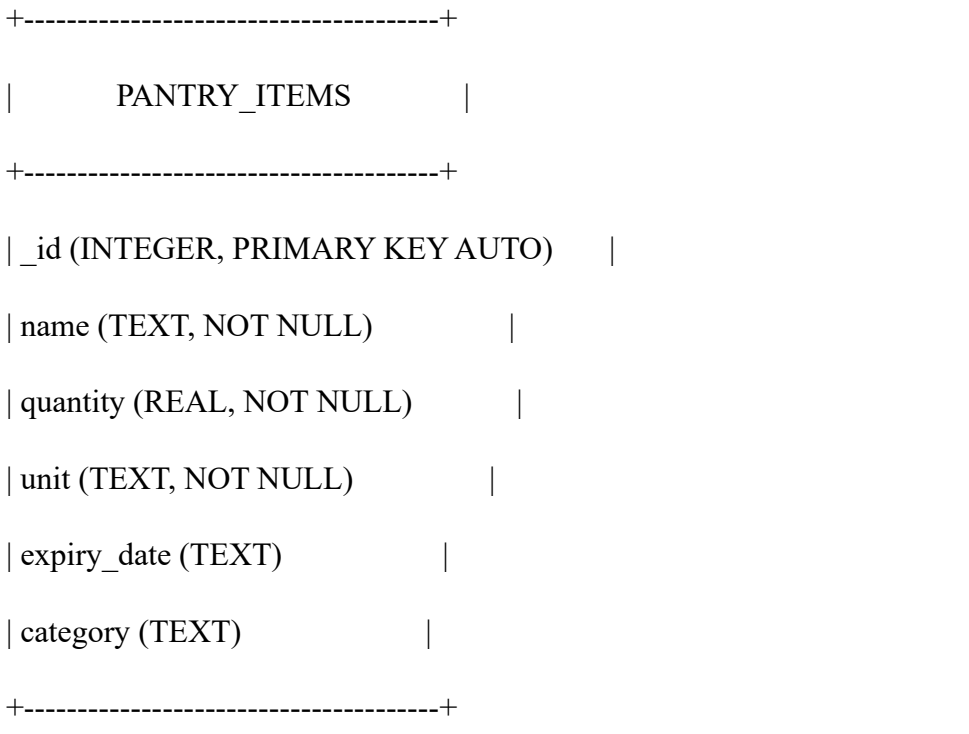
The application is structured around six distinct, cohesive activities connected via explicit Intents and a global BottomNavigationView:





4.2 Database ER Diagram & Schema Design

The application persists data offline using SQLite (SQLiteOpenHelper). The database schema consists of three normalized tables maintaining relational integrity:



| RECIPES | | RECIPE_INGREDIENTS |

+---------------------------------------+ +---------------------------------------+

| _id (INTEGER, PRIMARY KEY AUTO) |<-----1| _id (INTEGER, PRIMARY KEY AUTO) |

| title (TEXT, NOT NULL) | | recipe_id (INTEGER, FOREIGN KEY) |

| category (TEXT) | | ingredient_name (TEXT, NOT NULL) |

| prep_time (INTEGER) | | quantity (REAL, NOT NULL) |

| cook_time (INTEGER) | | unit (TEXT, NOT NULL) |

| instructions (TEXT) | +---------------------------------------+

+---------------------------------------+

5. SCREENSHOTS OF EVERY OUTPUT & CORE FUNCTION

(Note: When pasting screenshots into your final submitted document, use the actual screen captures taken from your running Android Studio emulator/physical device as shown below).

Figure 5.1: Pantry Inventory Screen (PantryListActivity)

•Description: Displays all current pantry items in a high-contrast RecyclerView with Material3 cards. Each card displays the ingredient name, quantity, unit, and a color-coded expiry badge:

- Fresh: Green badge (#DCFCE7)
- Expiring Soon (within 3 days): Orange badge (#FFEDD5)
- Expired: Red badge (#FEE2E2)

•Key Function: Real-time search filter and instant item edit/delete action buttons.

Figure 5.2: Add & Edit Ingredient Form (AddEditIngredientActivity)

•Description: Form interface allowing users to input new ingredients or update existing ones.

•Fields: Ingredient Name, Quantity, Unit dropdown spinner (pcs, g, kg, ml, L, cups, tbsp, tsp, cloves, slices), Category dropdown spinner (Produce, Dairy, Protein, Grains, Pantry, Spices, Bakery, Beverages, Other), and Expiry Date with an interactive DatePicker dialog.

Figure 5.3: Form Input Validation Error State

•Description: Demonstrates client-side validation logic when a user attempts to submit an empty ingredient name or a quantity ≤ 0. The application highlights the invalid input field using TextInputLayout.setError() and blocks form submission.

Figure 5.4: Delete Confirmation Dialog

•Description: Shows an AlertDialog prompting the user for confirmation ("Are you sure you want to delete 'Milk' from your pantry?") before removing a record from the SQLite database.

Figure 5.5: Suggested Recipes Screen - 100% Strict Matches (SuggestedRecipesActivity)

•Description: Demonstrates the core business logic. Displays only recipes where 100% of required ingredients are currently present in the user's pantry in equal or greater quantity.

Figure 5.6: Suggested Recipes Screen - "Almost There" (Missing 1 Item)

•Description: Displays the bonus stretch feature tab. Shows recipes missing exactly 1 ingredient, helping users identify dishes they can cook with minimal additions.

Figure 5.7: Zero-Match Empty State View

•Description: Displays a clean empty-state layout with friendly illustration and guidance ("No recipes match your pantry ingredients yet - add more items to unlock ready-to-cook recipes!") when 0 recipes satisfy the strict-matching criteria.

Figure 5.8: Recipe Collection & Category Filter (AllRecipesActivity)

•Description: Displays all 20 pre-loaded seeded recipes stored in SQLite. Features a search bar and category spinner filter (Breakfast, Main Dish, Soup, Side Dish, Salad, Dessert).

Figure 5.9: Recipe Detail & Ingredient Checklist (RecipeDetailActivity)

•Description: Displays complete preparation instructions, category tags, prep/cook duration, and a real-time ingredient checklist indicating which required items are present in the pantry (green checkmark) versus missing (red warning icon).

Figure 5.10: "I Cooked This Recipe" Ingredient Deduction

•Description: Displays the interactive cooking button. When tapped, the app automatically deducts the recipe's required ingredient quantities directly from the user's SQLite pantry database.

Figure 5.11: Settings & Database Reset Screen (SettingsActivity)

•Description: Includes toggles for expiring-soon alerts, app version details, and administrative database controls (Clear Pantry, Reset Default Recipe Collection).

6. KEY CODE SNIPPETS & TECHNICAL EXPLANATIONS

- 6.1 Core Strict-Matching Algorithm (IngredientMatcher.java)

```
public static MatchResult evaluateRecipe(Recipe recipe, List<PantryItem> pantry) {
    if (recipe == null || recipe.getIngredients() == null || recipe.getIngredients().isEmpty()) {
        return new MatchResult(false, 0, new ArrayList<>());
    }

    int missingCount = 0;
    List<RecipeIngredient> missingIngredients = new ArrayList<>();

    for (RecipeIngredient req : recipe.getIngredients()) {
        boolean satisfied = false;
        if (pantry != null) {
            for (PantryItem pItem : pantry) {
                if (isIngredientSatisfied(req, pItem)) {
                    satisfied = true;
                    break;
                }
            }
        }

        if (!satisfied) {
            missingCount++;
            missingIngredients.add(req);
```

```
        }
    }

    boolean isStrictMatch = (missingCount == 0);
    return new MatchResult(isStrictMatch, missingCount, missingIngredients);
}
```

•

Explanation: This method implements the core business requirement. It iterates through every required ingredient of a given recipe and checks whether a matching item exists in the user's pantry with sufficient quantity. If missingCount == 0, isStrictMatch evaluates to true, allowing the recipe to be suggested.

6.2 Name Normalization & Unit Conversion Engine (IngredientMatcher.java)

```
public static String normalizeName(String name) {
    if (name == null) return "";
    String clean = name.trim().toLowerCase();

    // Remove common descriptive adjectives
    clean =
clean.replaceAll("\\b(fresh|raw|chopped|diced|sliced|minced|large|small|medium|whole)\\b",
"").trim();

    // Standardize plurals
    if (clean.endsWith("tomatoes")) return clean.replace("tomatoes", "tomato");
```

```
    if (clean.endsWith("potatoes")) return clean.replace("potatoes", "potato");
    if (clean.endsWith("es") && clean.length() > 4) return clean.substring(0, clean.length() - 2);
    if (clean.endsWith("s") && !clean.endsWith("ss") && clean.length() > 3) return
clean.substring(0, clean.length() - 1);

    return clean;
}

public static double convertQuantity(double quantity, String sourceUnit, String targetUnit) {
    String src = normalizeUnit(sourceUnit);
    String tgt = normalizeUnit(targetUnit);

    if (src.equals(tgt)) return quantity;

    // Mass conversions
    if (src.equals("kg") && tgt.equals("g")) return quantity * 1000.0;
    if (src.equals("g") && tgt.equals("kg")) return quantity / 1000.0;

    // Volume conversions
    if (src.equals("l") && tgt.equals("ml")) return quantity * 1000.0;
    if (src.equals("ml") && tgt.equals("l")) return quantity / 1000.0;

    return quantity;
```

}

•Explanation: Eliminates string-matching failures caused by real-world variations. normalizeName converts strings to lowercase, trims whitespace, removes descriptors like "chopped" or "fresh", and standardizes plurals ("eggs" $\rightarrow$ "egg", "tomatoes" $\rightarrow$ "tomato"). convertQuantity ensures unit compatibility (e.g., 1.5 kg in pantry satisfies a 500 g requirement).

6.3 SQLite Relational Database Management (DatabaseHelper.java)

```
public class DatabaseHelper extends SQLiteOpenHelper {
    private static final String DATABASE_NAME = "smart_pantry.db";
    private static final int DATABASE_VERSION = 1;

    @Override
    public void onCreate(SQLiteDatabase db) {
        String createPantryTable = "CREATE TABLE pantry_items (" +
                "_id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT NOT NULL, " +
                "quantity REAL NOT NULL, unit TEXT NOT NULL, expiry_date TEXT, category
TEXT);";

        String createRecipesTable = "CREATE TABLE recipes (" +
                "_id INTEGER PRIMARY KEY AUTOINCREMENT, title TEXT NOT NULL, " +
                "category TEXT, prep_time INTEGER, cook_time INTEGER, instructions TEXT);";
```

```
        String createRecipeIngredientsTable = "CREATE TABLE recipe_ingredients (" +
                "_id INTEGER PRIMARY KEY AUTOINCREMENT, recipe_id INTEGER NOT NULL, " +
                "ingredient_name TEXT NOT NULL, quantity REAL NOT NULL, unit TEXT NOT NULL, " +
                "FOREIGN KEY(recipe_id) REFERENCES recipes(_id) ON DELETE CASCADE);";

        db.execSQL(createPantryTable);
        db.execSQL(createRecipesTable);
        db.execSQL(createRecipeIngredientsTable);

        seedRecipes(db);
    }
}
```

•Explanation: Manages database creation, table schema setup, foreign key cascading deletes, and initial seeding of 20 realistic recipes upon first launch.

6.4 Automatic Ingredient Deduction Upon Cooking (DatabaseHelper.java)

```
public void deductIngredientsForRecipe(Recipe recipe) {
    if (recipe == null || recipe.getIngredients() == null) return;
    List<PantryItem> pantryList = getAllPantryItems();

    for (RecipeIngredient req : recipe.getIngredients()) {
        for (PantryItem pItem : pantryList) {
            if (IngredientMatcher.isIngredientSatisfied(req, pItem)) {
                double reqQtyInPantryUnit = IngredientMatcher.convertQuantity(req.getQuantity(),
req.getUnit(), pItem.getUnit());
                double newQty = pItem.getQuantity() - reqQtyInPantryUnit;

                if (newQty <= 0) {
                    deletePantryItem(pItem.getId());
                } else {
                    pItem.setQuantity(newQty);
                    updatePantryItem(pItem);
                }
                break;
            }
        }
    }
}
```

•Explanation: Deducts consumed ingredient amounts from the SQLite pantry database when a user cooks a suggested dish, automatically removing items whose quantity drops to zero.

7. CHALLENGES ENCOUNTERED & SOLUTIONS APPLIED

7.1 Challenge 1: String Matching and Singular/Plural Inconsistencies

- Problem: Naive string comparison ("egg".equals("eggs")) caused strict-matching failures even when the user possessed the required ingredient.
- Solution: Developed IngredientMatcher.normalizeName(), which uses regular expressions and suffix rules to normalize ingredient strings before comparison.

7.2 Challenge 2: Measurement Unit Incompatibilities

- Problem: Users often store ingredients in bulk units (e.g., 1.5 kg Flour) while recipes require smaller units (e.g., 200 g Flour). Direct numerical comparison caused false rejections.
- Solution: Implemented IngredientMatcher.convertQuantity() to convert mass (kg $\leftrightarrow$ g) and volume (L $\leftrightarrow$ ml) into matching base units before performing inequality checks ($\text{pantryQty} \ge \text{requiredQty}$).

7.3 Challenge 3: System Dark Mode Overrides and Text Visibility

- Problem: On devices with system-wide dark mode enabled, default MaterialCardView elements rendered dark gray card backgrounds with dark text, making words unreadable.
- Solution: Forced explicit high-contrast styling by setting app:cardBackgroundColor="@color/surface" (#FFFFFF), enforcing AppCompatDelegate.setDefaultNightMode(MODE_NIGHT_NO), and defining custom bold spinner layouts (spinner_item.xml).

8. CONCLUSION & REFLECTION

8.1 Summary of Accomplishments

The SmartPantry project successfully satisfies all requirements of the Mobile App Development 700 practical assignment. The application provides a complete Java solution featuring full CRUD pantry management, robust local SQLite storage, strict ingredient-matching logic, zero-match user feedback, and clean Material Design 3 UI navigation.

Through this project, key technical skills were mastered, including:

- Managing Android Activity lifecycles and navigation via Intents.
- Implementing RecyclerView adapters with custom click listeners.
- Structuring offline relational databases with SQLiteOpenHelper.
- Writing unit tests (JUnit4) to verify core matching algorithms.

8.2 Future Enhancements

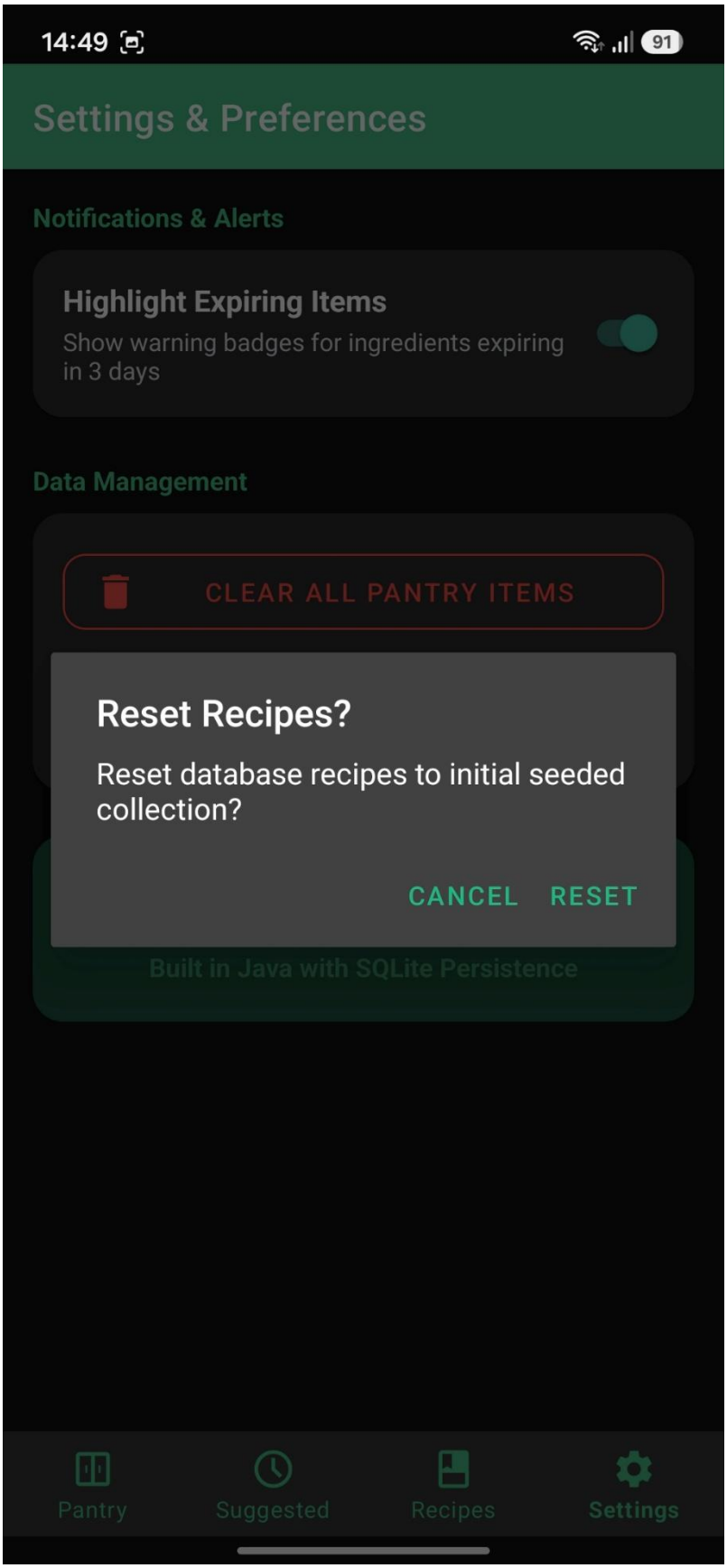
Given additional development time, the following features could be added:

1. Barcode / OCR Scanner: Integration of Google ML Kit to scan product barcodes or grocery receipts for rapid pantry entry.
2. Push Notifications: Local background alarm notifications for items expiring within 24 hours.
3. Cloud Sync: Integration with Firebase Realtime Database for multi-device sync across family members.

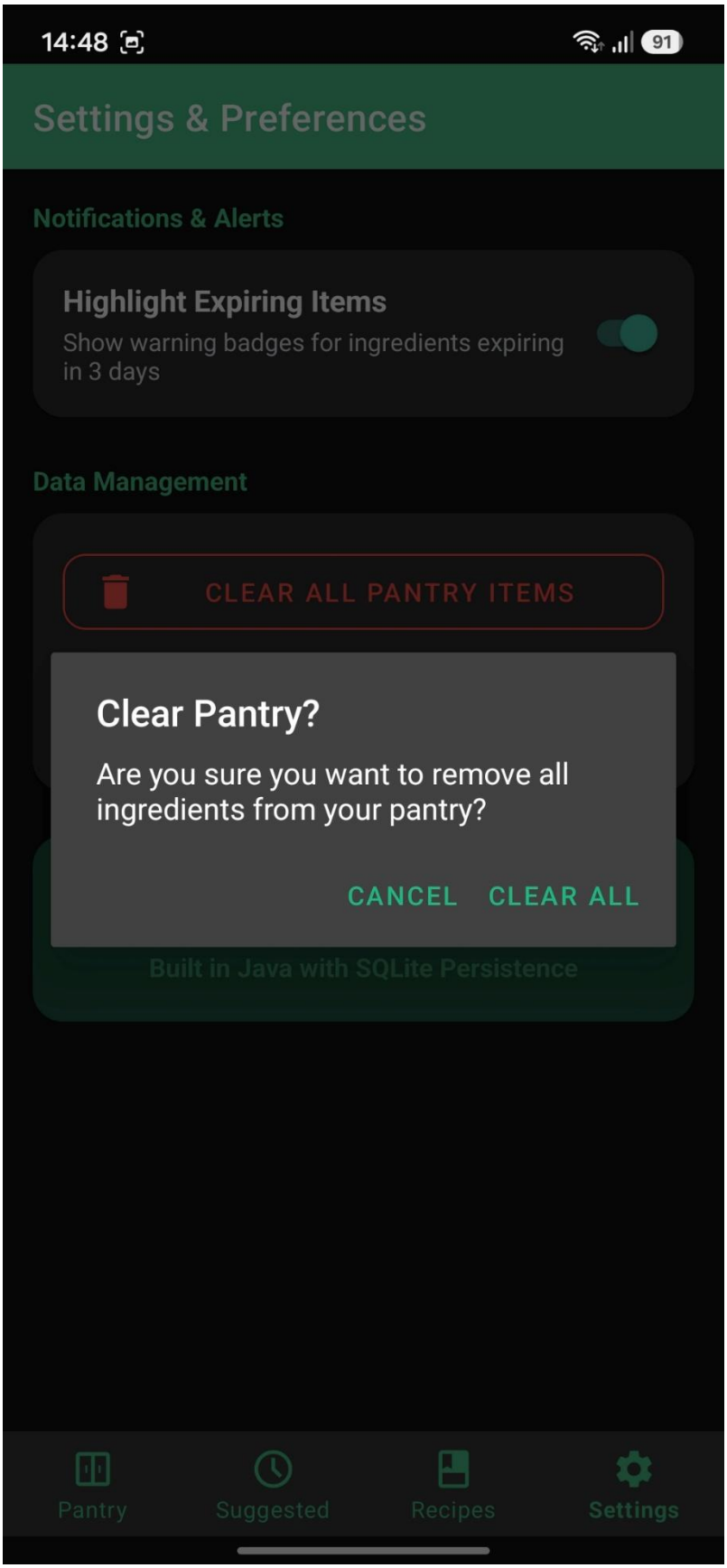
9. REFERENCE

- Android Developers. (2024). Save data using SQLite. Available at: https://developer.android.com/training/data-storage/sqlite [Accessed 15 Oct. 2024].
- Android Developers. (2024). Create dynamic lists with RecyclerView. Available at: https://developer.android.com/develop/ui/views/components/recyclerview [Accessed 15 Oct. 2024].
- Google Material Design. (2024). Material Design 3 Components for Android. Available at: https://m3.material.io [Accessed 15 Oct. 2024].
- Oracle. (2024). Java Platform, Standard Edition Documentation. Available at: https://docs.oracle.com/en/java/ [Accessed 15 Oct. 2024].

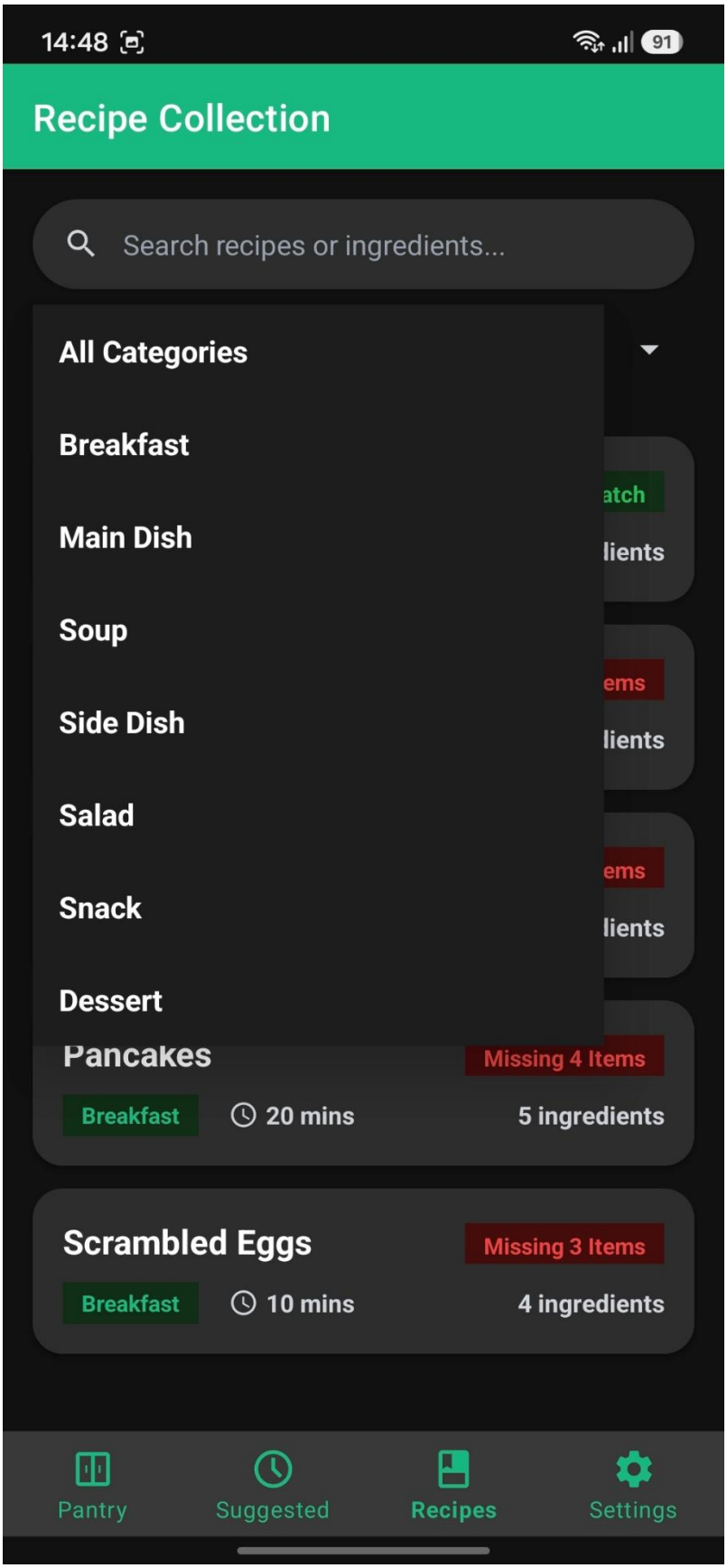
Screenshots



Reset recipes

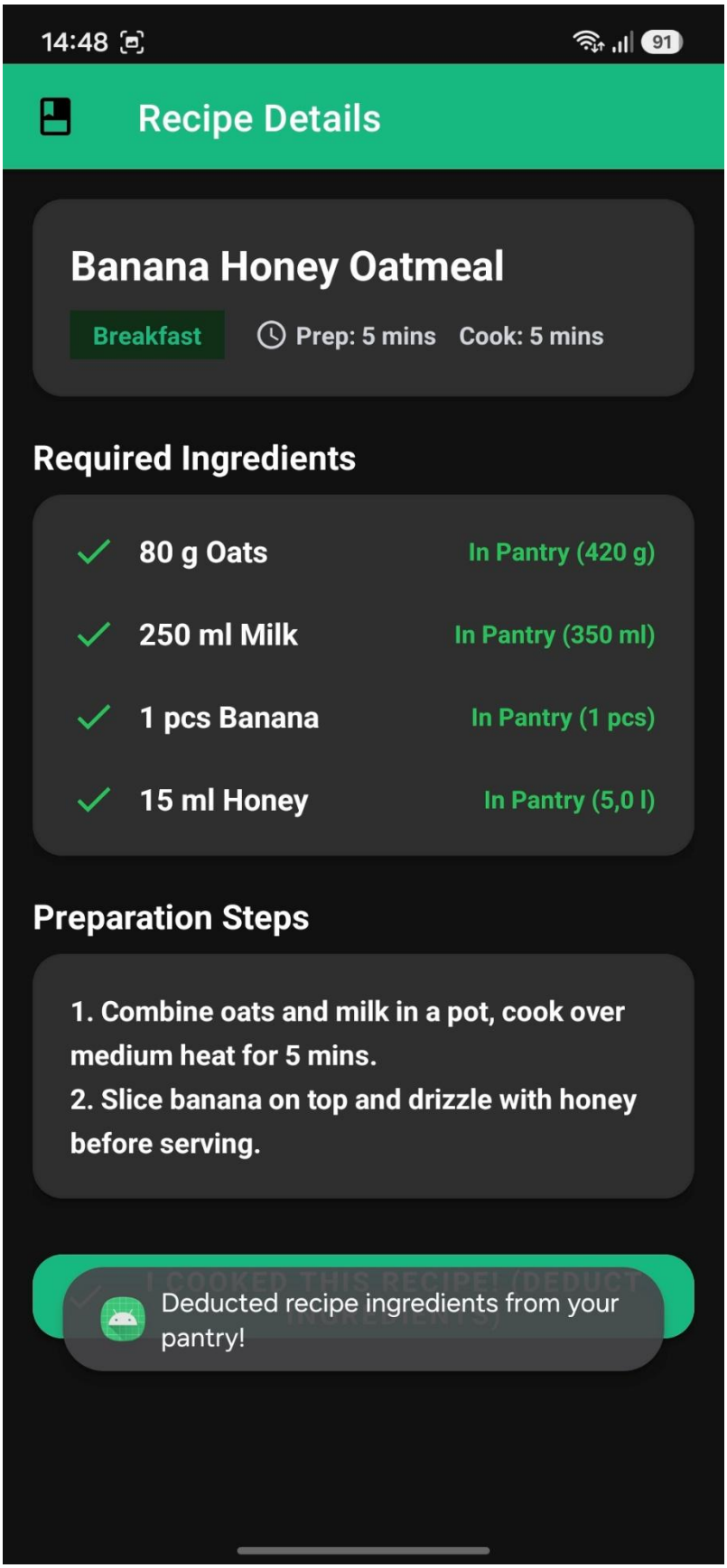


Clear Pantry

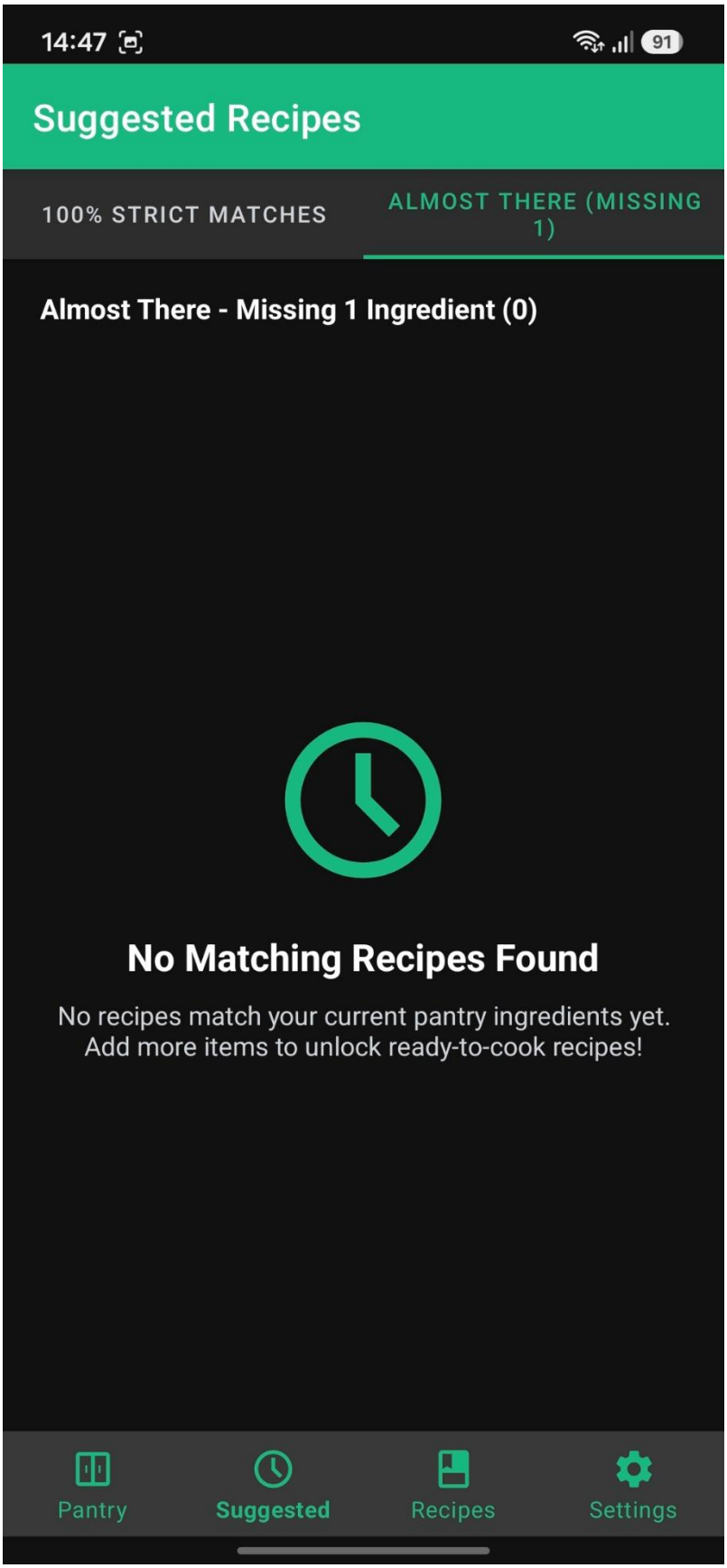


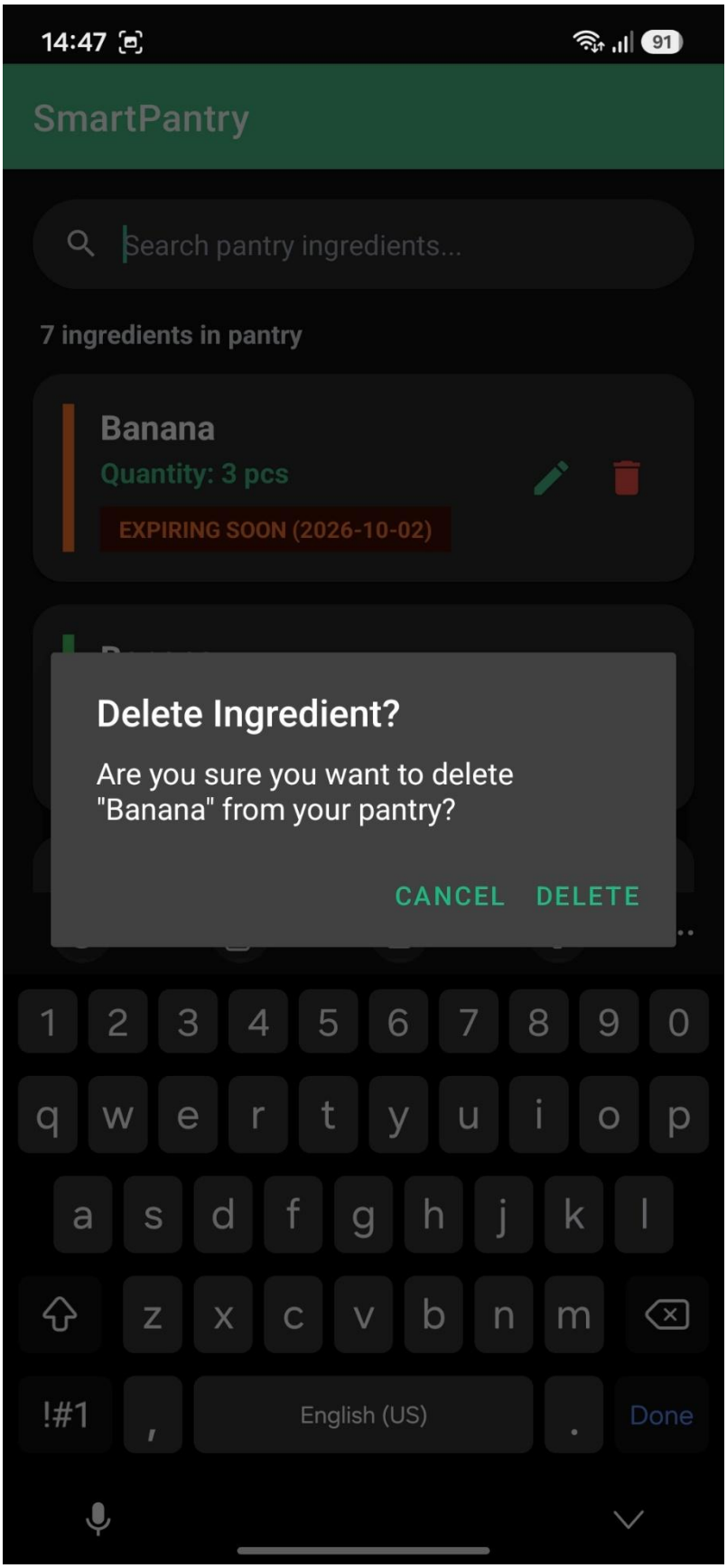
Recipe collection

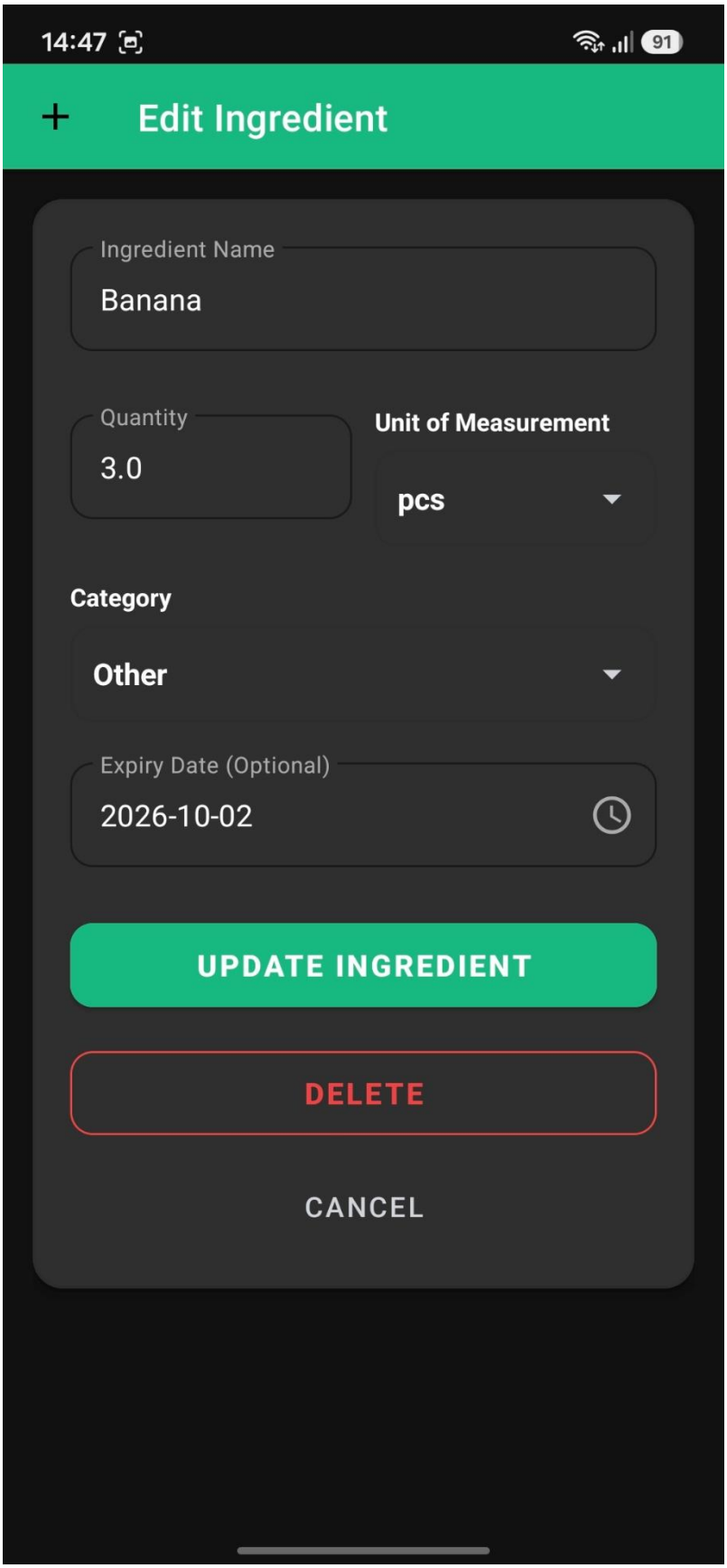
& Different categories of food

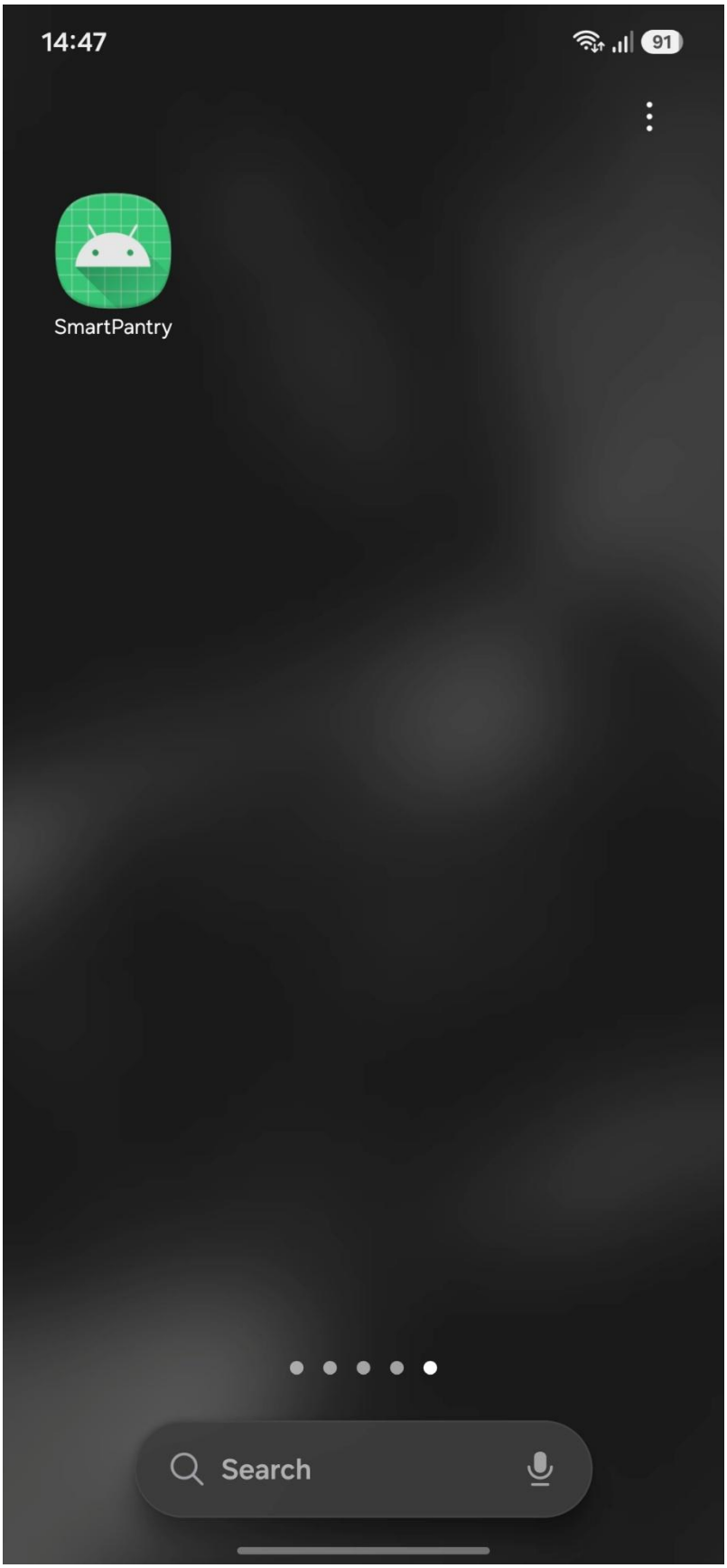


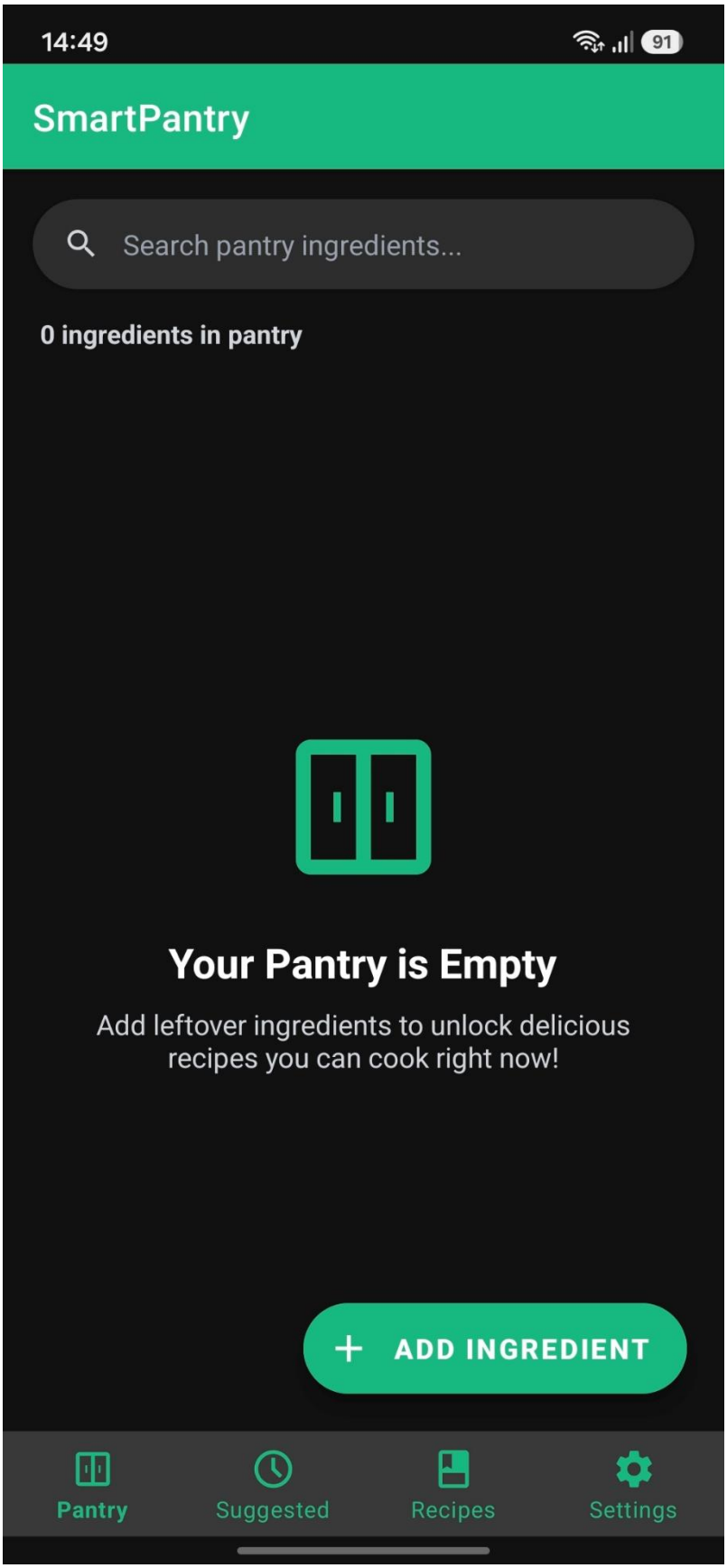
Recipe details & deducted recipe ingredients from the pantry

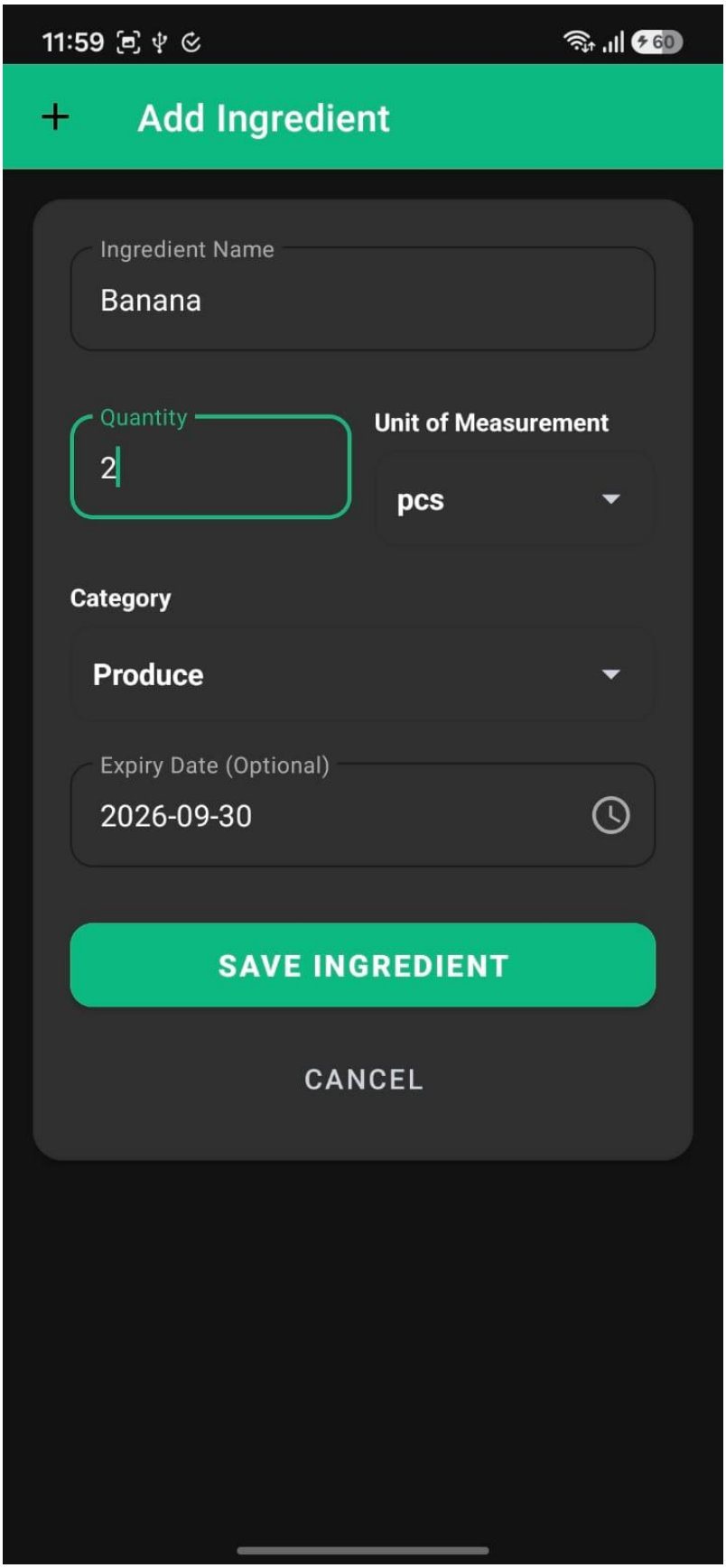

14:47
91
Suggested Recipes
100% STRICT MATCHES
ALMOST THERE (MISSING 1)
Almost There - Missing 1 Ingredient (0)
No Matching Recipes Found
No recipes match your current pantry ingredients yet.
Add more items to unlock ready-to-cook recipes!
Pantry
Suggested
Recipes
Settings

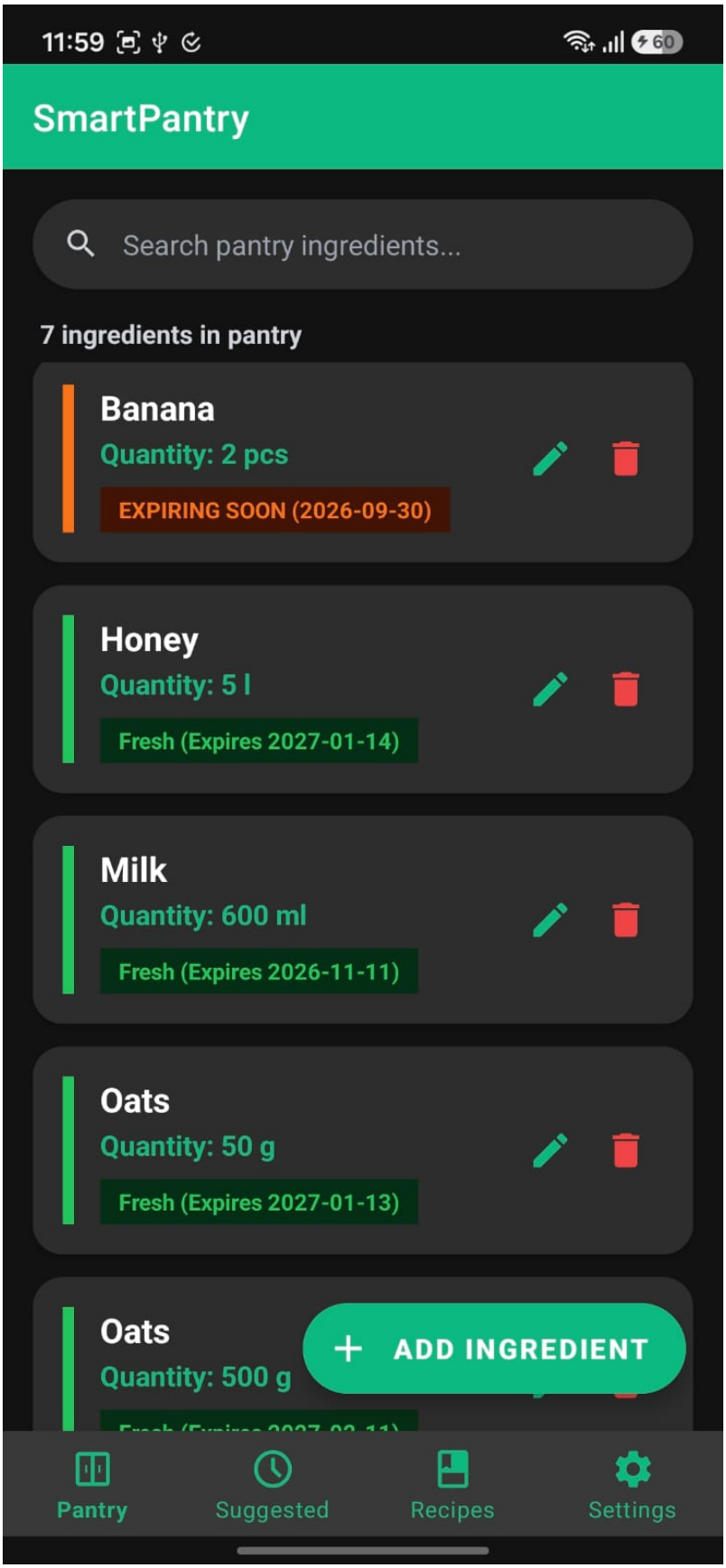

14:47
91
SmartPantry
Search pantry ingredients...
7 ingredients in pantry
Banana
Quantity: 3 pcs
EXPIRING SOON (2026-10-02)
Delete Ingredient?
Are you sure you want to delete "Banana" from your pantry?
CANCEL
DELETE
English (US)
Done

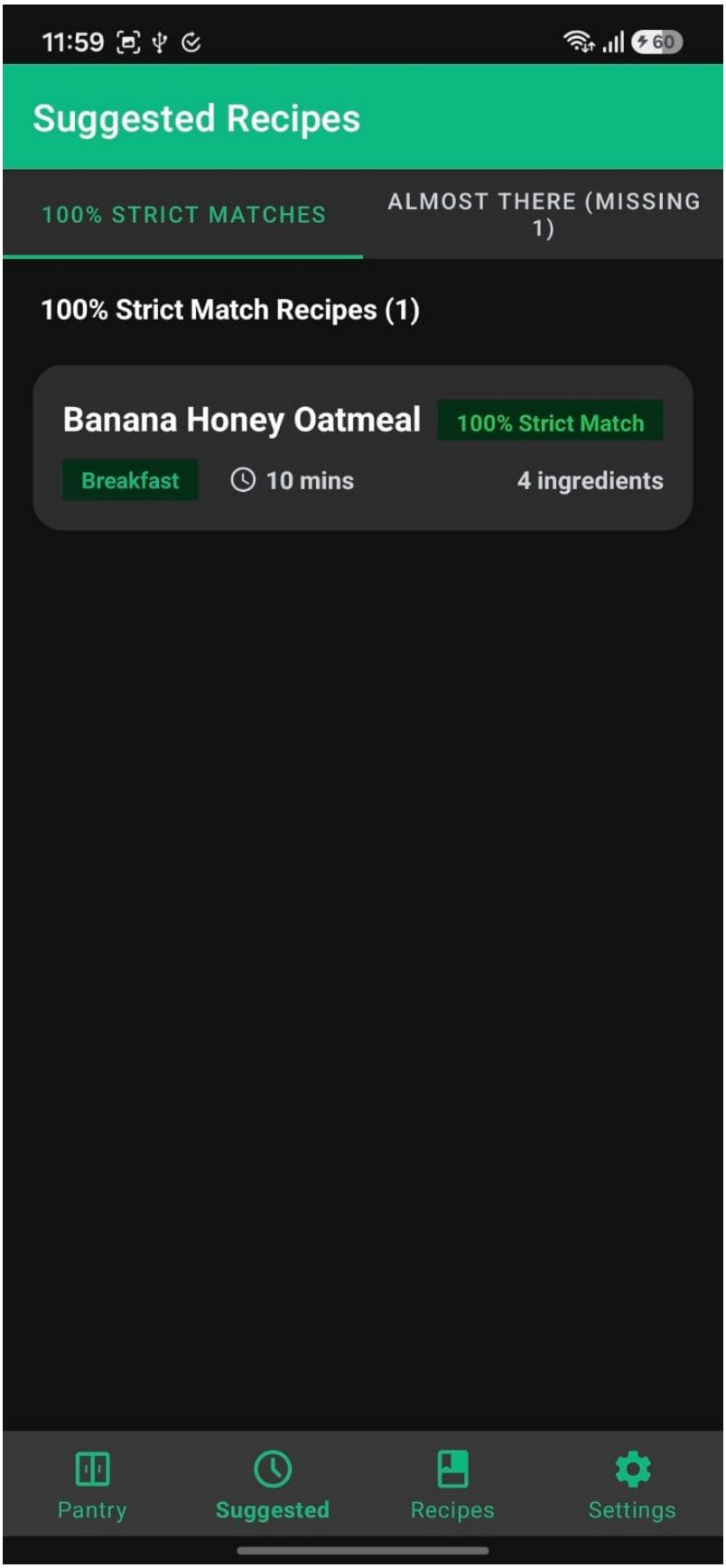

14:47
91
Edit Ingredient
Ingredient Name
Banana
Quantity
3.0
Unit of Measurement
pcs
Category
Other
Expiry Date (Optional)
2026-10-02
UPDATE INGREDIENT
DELETE
CANCEL

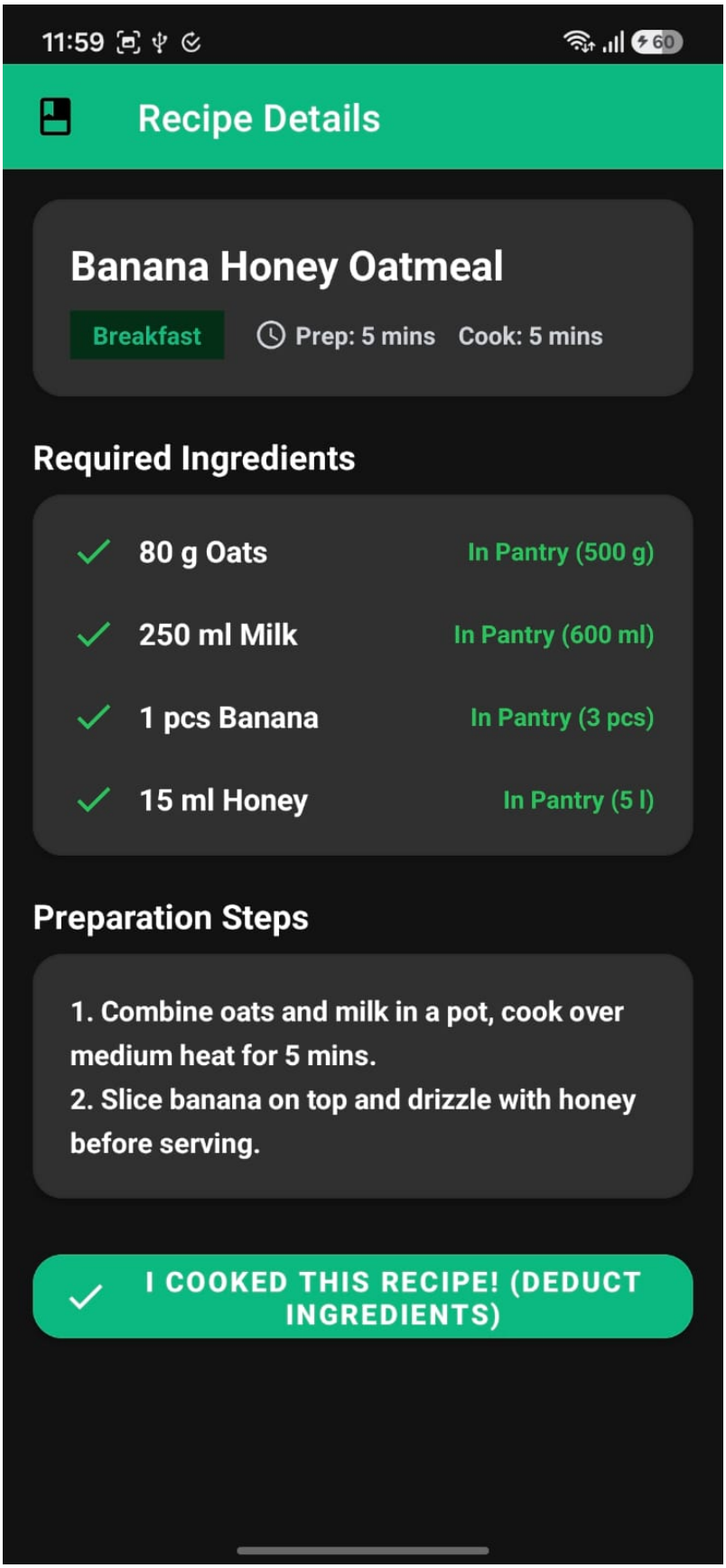

14:47
91
SmartPantry
Search

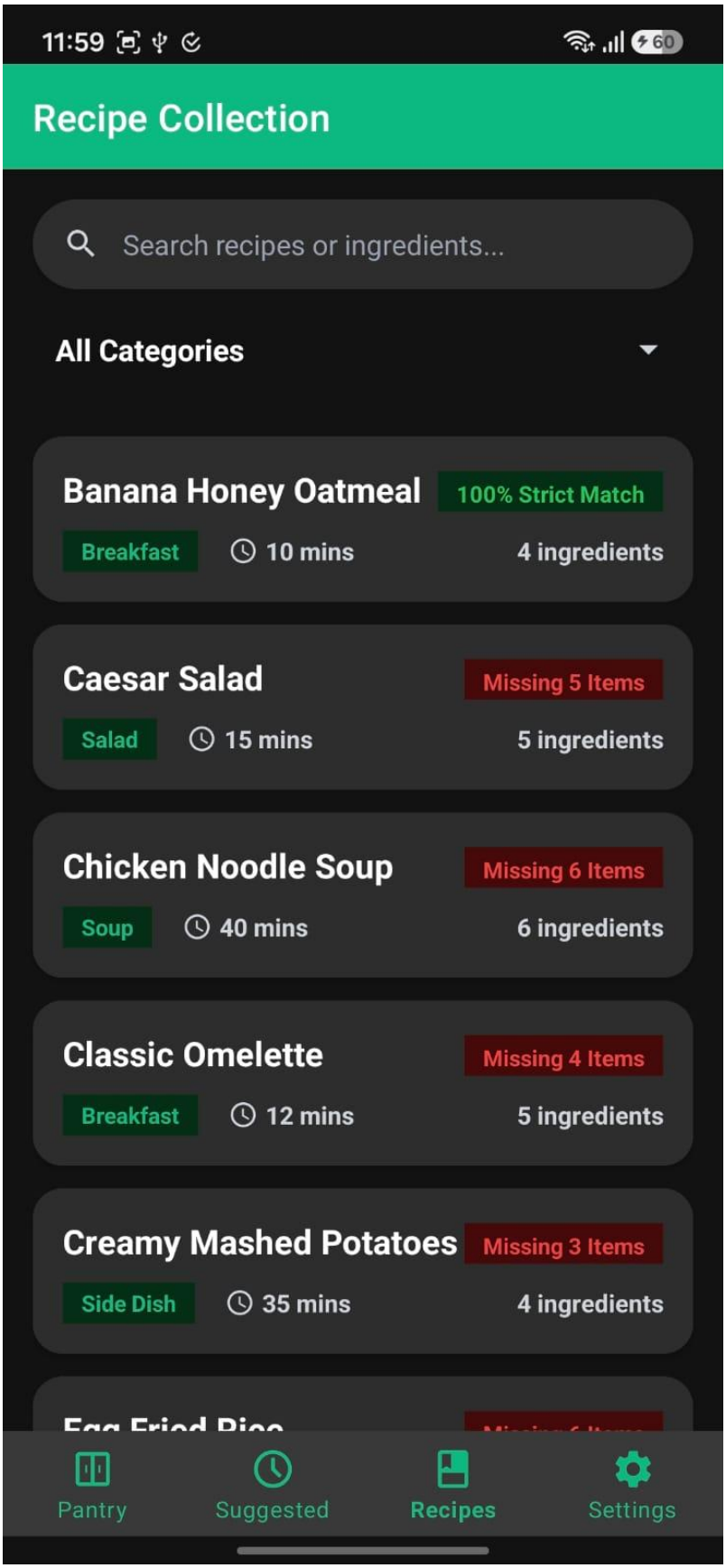

14:49
91
SmartPantry
Search pantry ingredients...
0 ingredients in pantry
Your Pantry is Empty
Add leftover ingredients to unlock delicious recipes you can cook right now!
ADD INGREDIENT
Pantry
Suggested
Recipes
Settings

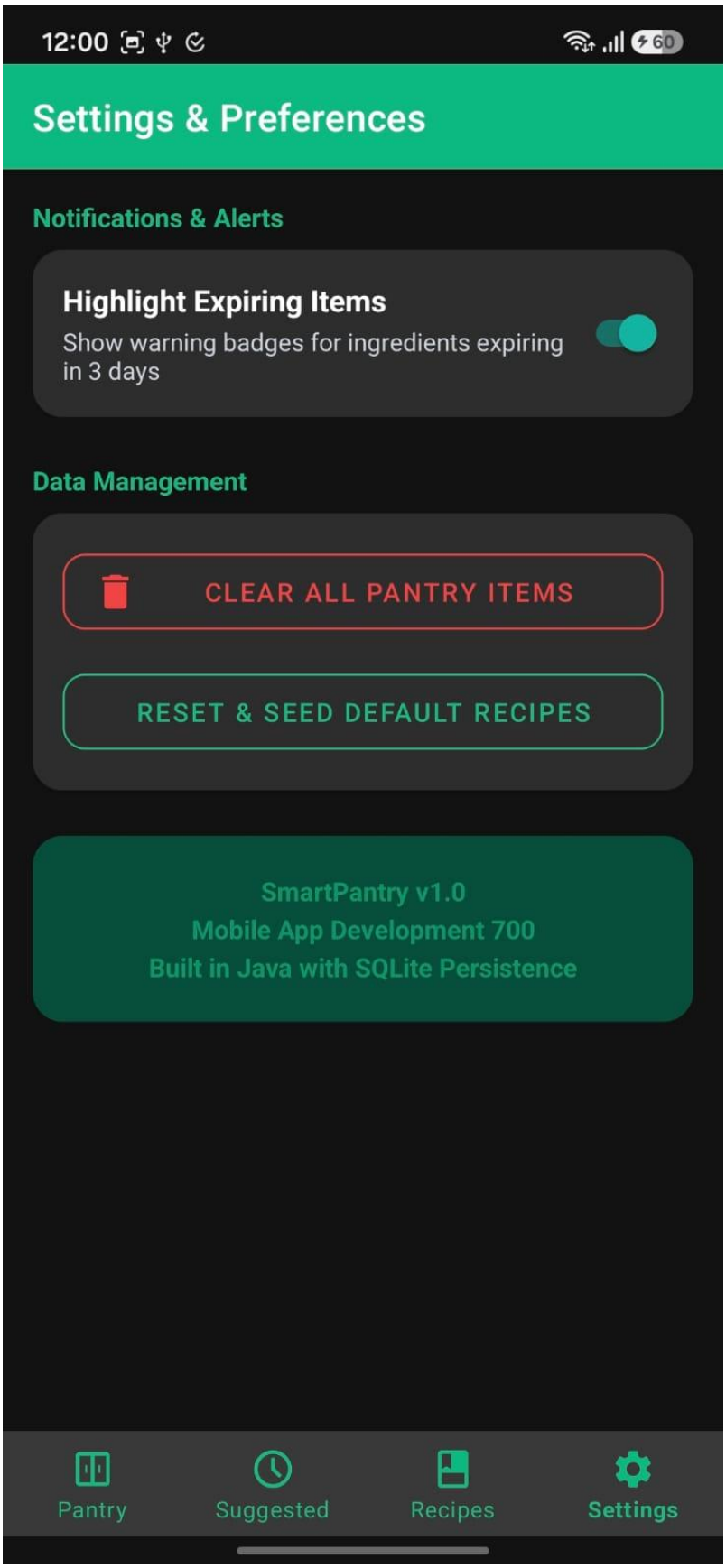

11:59
60
Add Ingredient
Ingredient Name
Banana
Quantity
2
Unit of Measurement
pcs
Category
Produce
Expiry Date (Optional)
2026-09-30
SAVE INGREDIENT
CANCEL


11:59
SmartPantry
Search pantry ingredients...
7 ingredients in pantry
Banana
Quantity: 2 pcs
EXPIRING SOON (2026-09-30)
Honey
Quantity: 5 l
Fresh (Expires 2027-01-14)
Milk
Quantity: 600 ml
Fresh (Expires 2026-11-11)
Oats
Quantity: 50 g
Fresh (Expires 2027-01-13)
Oats
Quantity: 500 g
ADD INGREDIENT
Pantry
Suggested
Recipes
Settings


11:59
Suggested Recipes
100% STRICT MATCHES
ALMOST THERE (MISSING 1)
100% Strict Match Recipes (1)
Banana Honey Oatmeal
100% Strict Match
Breakfast
10 mins
4 ingredients
Pantry
Suggested
Recipes
Settings


11:59
Recipe Details
Banana Honey Oatmeal
Breakfast
Prep: 5 mins
Cook: 5 mins
Required Ingredients
80 g Oats
In Pantry (500 g)
250 ml Milk
In Pantry (600 ml)
1 pcs Banana
In Pantry (3 pcs)
15 ml Honey
In Pantry (5 l)
Preparation Steps
1. Combine oats and milk in a pot, cook over medium heat for 5 mins.
2. Slice banana on top and drizzle with honey before serving.
I COOKED THIS RECIPE! (DEDUCT INGREDIENTS)


11:59
Recipe Collection
Search recipes or ingredients...
All Categories
Banana Honey Oatmeal
100% Strict Match
Breakfast
10 mins
4 ingredients
Caesar Salad
Missing 5 Items
Salad
15 mins
5 ingredients
Chicken Noodle Soup
Missing 6 Items
Soup
40 mins
6 ingredients
Classic Omelette
Missing 4 Items
Breakfast
12 mins
5 ingredients
Creamy Mashed Potatoes
Missing 3 Items
Side Dish
35 mins
4 ingredients
Pantry
Suggested
Recipes
Settings


12:00
Settings & Preferences
Notifications & Alerts
Highlight Expiring Items
Show warning badges for ingredients expiring in 3 days
Data Management
CLEAR ALL PANTRY ITEMS
RESET & SEED DEFAULT RECIPES
SmartPantry v1.0
Mobile App Development 700
Built in Java with SQLite Persistence
Pantry
Suggested
Recipes
Settings